Express JS

What is Express?

- Express.js (or simply Express) is the most popular Node.js web application framework, designed for building web applications and APIs.

```
const express = require('express');
const app = express();
const port = 8080;

// Define a route for GET requests to the root URL
app.get('/', (req, res) => {
  res.send('Hello World from Express!');
});

// Start the server
app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});
```

Brief History of Express

Express.js, the standard **server framework** for Node.js, was created to **simplify web application** and **API development**.

- **Creation and Initial Release:** Express.js was founded by the prolific open-source developer TJ Holowaychuk. The first version was released on November 16, 2010.
- **Acquisition by StrongLoop and IBM:** In June 2014, Holowaychuk sold to StrongLoop company, in September 2015, IBM acquired StrongLoop.
- **Stewardship by the Node.js Foundation:** In January 2016, IBM announced that it would place Express.js under the stewardship of the Node.js Foundation (now part of the OpenJS Foundation), which helped ensure its long-term sustainability and community-driven governance.
- **Evolution and Version 4.x:** the long-standing stable version for many years.
- **Version 5.x Release:** Version 5 was released in October 2024, becoming the default on npm in early 2025.

Today, Express.js has a large and active community, an extensive ecosystem of middleware, and is used by major companies like Netflix, PayPal, and Uber to power their web applications and APIs.

Routing

- Routing refers to how an application's **endpoints** (URIs) respond to client requests.

- You define routing using methods of the Express app object that correspond to HTTP methods; for example, **app.get()** to handle **GET** requests and **app.post()** to handle **POST** requests.

Node JS Ecosystem

Node.js Runtime – Executes JavaScript code server-side outside a browser.

Package Manager – npm or yarn for managing dependencies and modules.

Express.js – Minimalist web framework for building APIs and applications.

Frameworks – Tools like Nest.js, Koa, or Fastify for structured development.

Database Integration – Libraries connecting to SQL, NoSQL, and other data stores.

Real-time Communication – WebSocket support for instant bidirectional communication.

Testing and Quality Assurance – Tools like Jest, Mocha for reliable code validation.

Task Runners and Build Tools – Automate development workflows and bundling.

Deployment and Hosting – Platforms and practices for launching Node.js applications.

Community and Resources – Vibrant ecosystem with extensive documentation and support.

Node.js Libraries – Reusable modules for adding functionality to applications.

Middleware in Express JS

Middleware in Express.js are functions that run during the request–response lifecycle to process requests, modify responses, and control application flow.

- Express is a routing and middleware web framework that has minimal functionality of its own.
- An Express application is essentially a series of **middleware** function calls.
- Middleware functions are functions that have access to
  - **the request object (req),**
  - **the response object (res),**
  - **and the next middleware function in the application's request-response cycle.**
- The next middleware function is commonly denoted by a variable named next.

Middleware Working

- Executes custom logic for each request.
- Can read or modify req and res.
- May send a response and end the cycle.
- Uses next() to pass control to the next middleware.

Types of Middleware

- ExpressJS offers different types of middleware and you should choose the middleware based on functionality required.

1. **Application-level Middleware**

- Application-level middleware runs across the entire Express application, handling common logic for all incoming requests.
  - Registered using app.use() or app.METHOD().
  - Executes for all routes and HTTP methods.
  - Commonly used for logging, authentication, body parsing, and headers.

```
app.use(express.json()); // Parses JSON data for every incoming request
app.use((req, res, next) => {
  console.log('Request received:', req.method, req.url);
  next();
});
```

The following figure shows the elements of a middleware function call:

```
var express = require('express');
var app = express();

app.get('/', function(req, res, next) {
  next();
})

app.listen(3000);
```

- HTTP method for which the middleware function applies.
- Path (route) for which the middleware function applies.
- The middleware function.
- Callback argument to the middleware function, called "next" by convention.
- HTTP response arg. to the middleware function, called "res" by convention.
- HTTP request argument to the middleware function, called "req" by convention.

Types of Middleware

2. Router-level Middleware
3. Error-handling Middleware
4. Built-in Middleware
5. Third-party Middleware

Example: Router-level Middleware

```
const router = express.Router();

router.use((req, res, next) => {
  console.log('Router-specific middleware');
  next();
});

router.get('/dashboard', (req, res) => {
  res.send('Dashboard Page');});
app.use('/user', router); // The middleware
applies only to routes under "/user"
```

Example: Error-handling Middleware

```
app.use((err, req, res, next) => {
  console.error(err.stack); // Log the error stack
  res.status(500).send('Something went wrong!');
});
```

Install node JS, express and thunder client extension in VSCode

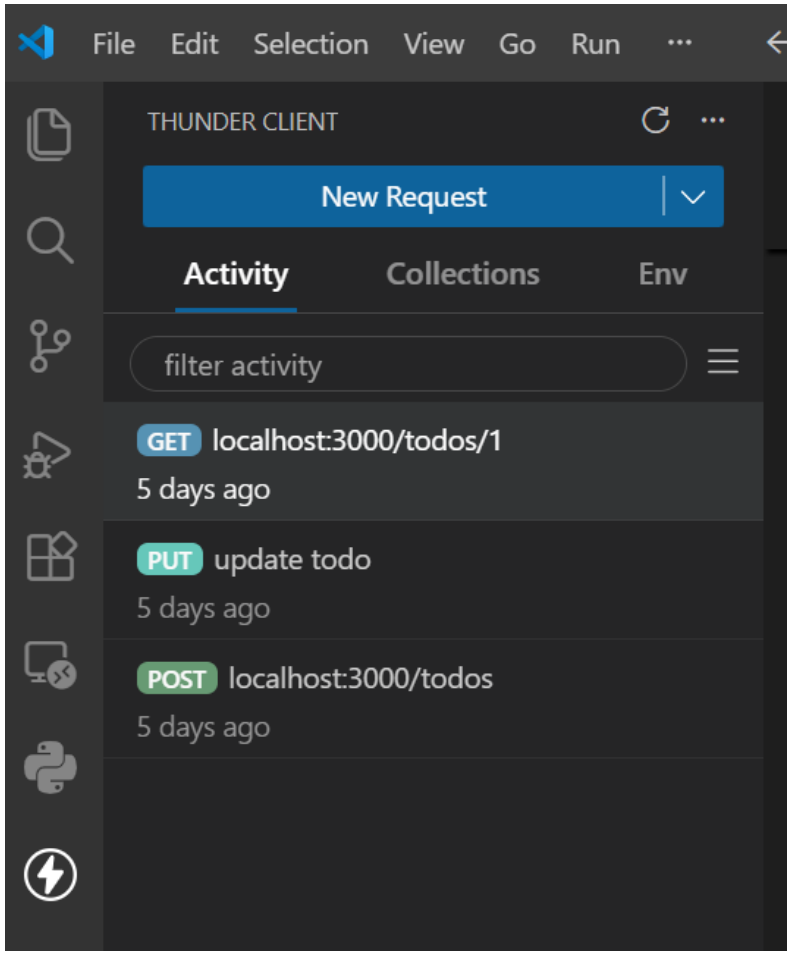
Installing Express

To use Express in your Node.js application, you first need to install it:

```
npm install express
```

To install Express and save it in your package.json dependencies:

```
npm install express --save
```



File Edit Selection View Go Run ··· tasktrek

EXPLORER

TASKTREK
- node_modules
- index.js
- package-lock.json
- package.json

index.js | package.json

package.json > ...

```
1  {
2    "name": "tasktrek",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
     ▷Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs",
13   "dependencies": {
14     "express": "^5.1.0"
15   }
16 }
17
```

Basic Routing

- app.get() - Handle GET requests
- app.post() - Handle POST requests
- app.put() - Handle PUT requests
- app.delete() - Handle DELETE requests

File Edit Selection View Go Run ··· myExpressExmp

EXPLORER

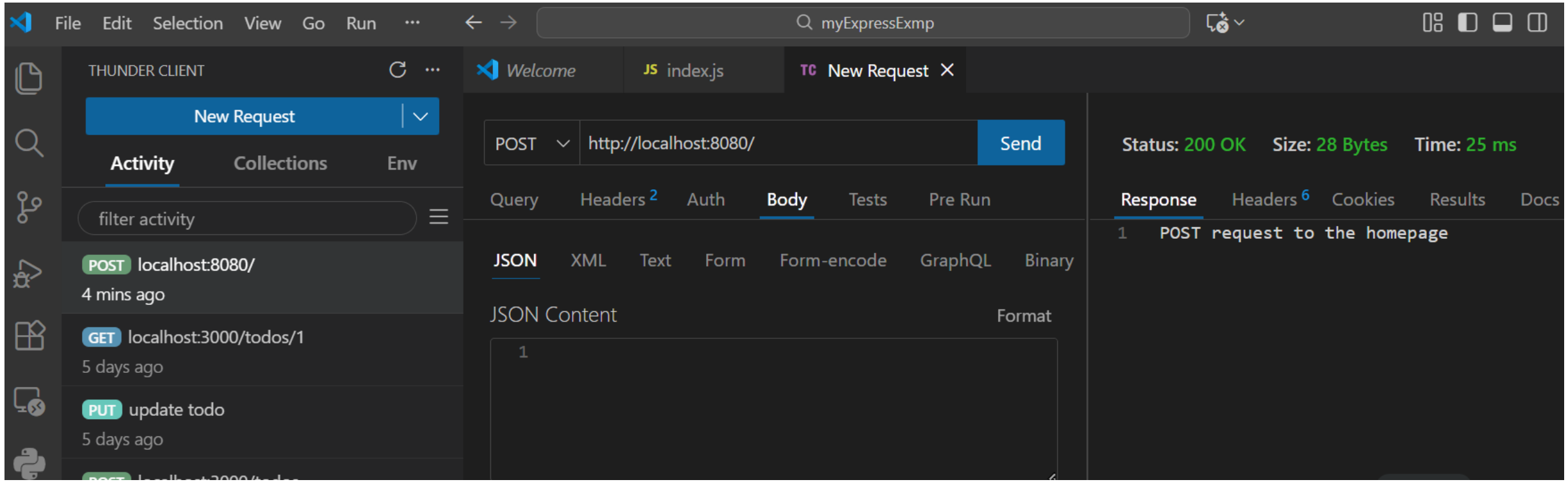
MYEXPRESSEXMP
- node_modules
- index.js
- package-lock.json
- package.json

Welcome | index.js | New Request

index.js > ...

```javascript
const express = require('express');
const app = express();
const port = 8080;

// Respond to GET request on the root route
app.get('/', (req, res) => {
  res.send('GET request to the homepage');
});

// Respond to POST request on the root route
app.post('/', (req, res) => {
  res.send('POST request to the homepage');
});

// Respond to GET request on the /about route
app.get('/about', (req, res) => {
  res.send('About page');
});

// Start the server
app.listen(port, () => {
  console.log(`Example app listening at http://localhost:${port}`);
});X
```

Thunder Client (POST event)



Express.js — from installation to execution

Prerequisites (One-time setup)

- Before touching Express, make sure these are installed:
  - Node.js (comes with npm)
  - Visual Studio Code
  - Basic JS knowledge
- Check versions in VS Code terminal:
  - node -v
  - npm -v

Create Project Folder & Open in VS Code

```
mkdir express-demo
cd express-demo
code .
```

Initialize Node Project

- npm init –y
- This creates: package.json
- **Why important?**
  - Stores project metadata
  - Manages dependencies
  - Defines start scripts
- Install Express.js
  - **npm install express**
  - Now your folder looks like:

```
express-demo
 ├── node_modules/
 ├── package.json
 ├── package-lock.json
```

Code

- Create a new file: app.js
- Add this code

```
const express = require('express');
const app = express();

// route
app.get('/', (req, res) => {
  res.send('Hello Express!');
});

// server
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

- Run Express App
  - node app.js
- Server running on http://localhost:3000
- Open browser:
  - http://localhost:3000
  - Hello Express!

Express.js MVC Structure (Model, View, Controller)

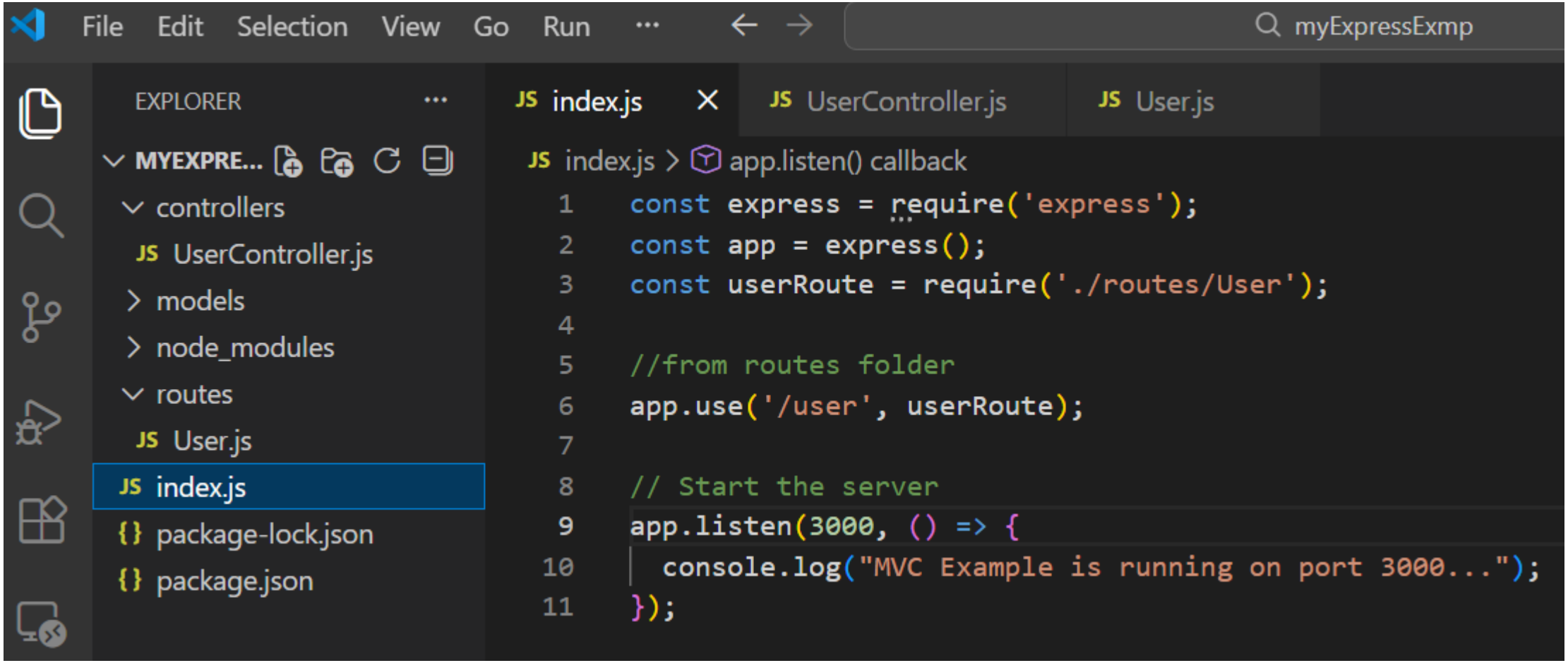
1. **Model**
   - **What it is:** Represents the application's data structure and business logic.
   - **In Express.js:** Typically interacts with a database (via libraries like Mongoose or Sequelize), defines data schemas, and contains data validation and manipulation rules.
   - **Example:** A `User` model that defines what a user is (name, email, password) and can save or retrieve users from the database.
2. **View**
   - **What it is:** The presentation layer, responsible for how the data is displayed to the user.
   - **In Express.js:** Often implemented with templating engines like **EJS**, **Pug (Jade)**, or **Handlebars**. It renders dynamic HTML by injecting data received from the controller.
   - **Example:** An `index.ejs` file that takes a list of blog posts and generates the HTML to display them.

3. **Controller**
   - **What it is:** The intermediary that handles user input (from routes), interacts with the Model to process data, and then passes that data to the View for rendering (or sends a direct response like JSON for an API).
   - **In Express.js:** Contains the route handler functions (like `getAllUsers`, `createPost`). It receives the request (`req`), works with the Model, and sends a response (`res`).
   - **Example:** A `userController.js` with a `getUserProfile` function that fetches a user from the `User` model and renders the `profile.ejs` view with that user's data.

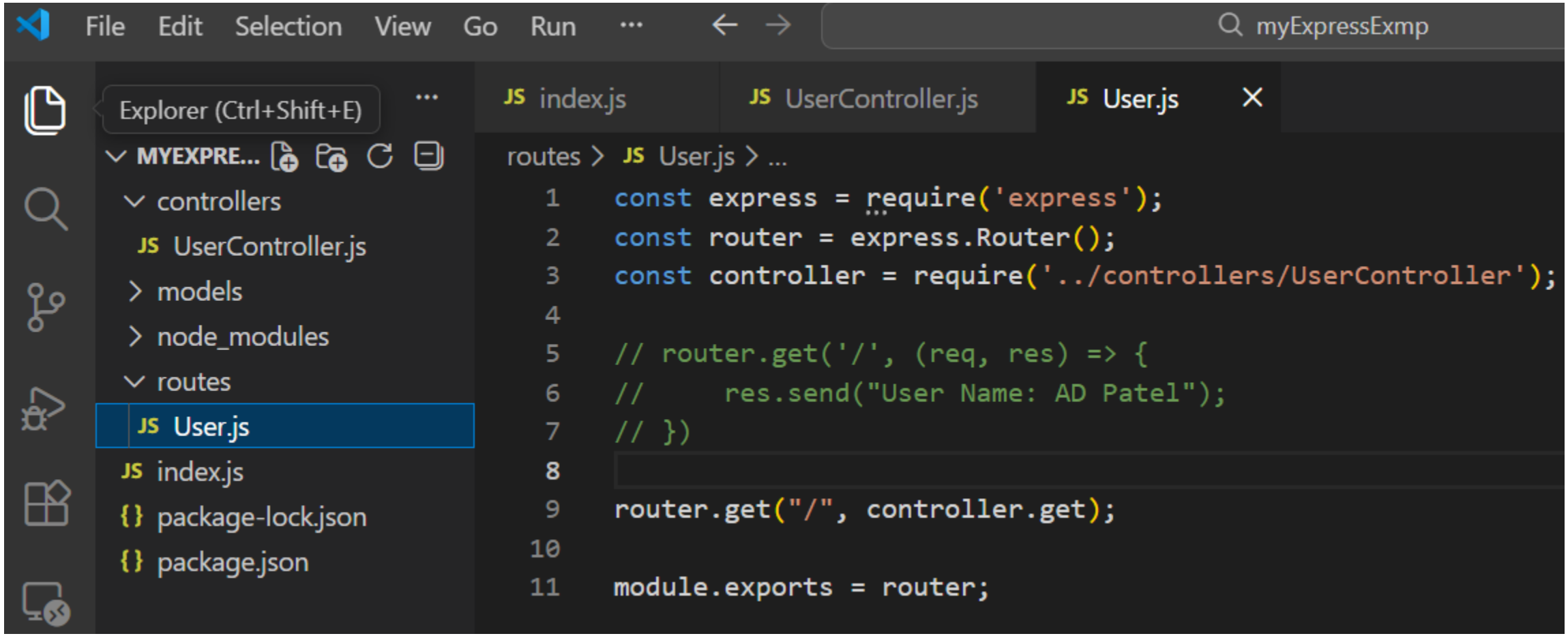
Typical Flow (Request Cycle):

1. **Route** maps a URL (e.g., /posts) to a specific **Controller** function.
2. The **Controller** function calls the **Model** (e.g., Post.getAll()).
3. The **Model** interacts with the **database** and returns data to the **Controller**.
4. The **Controller** takes that data and passes it to a **View** (e.g., res.render('posts', {posts: data})).
5. The **View** engine renders the **final HTML** and sends it back to the user's **browser**.

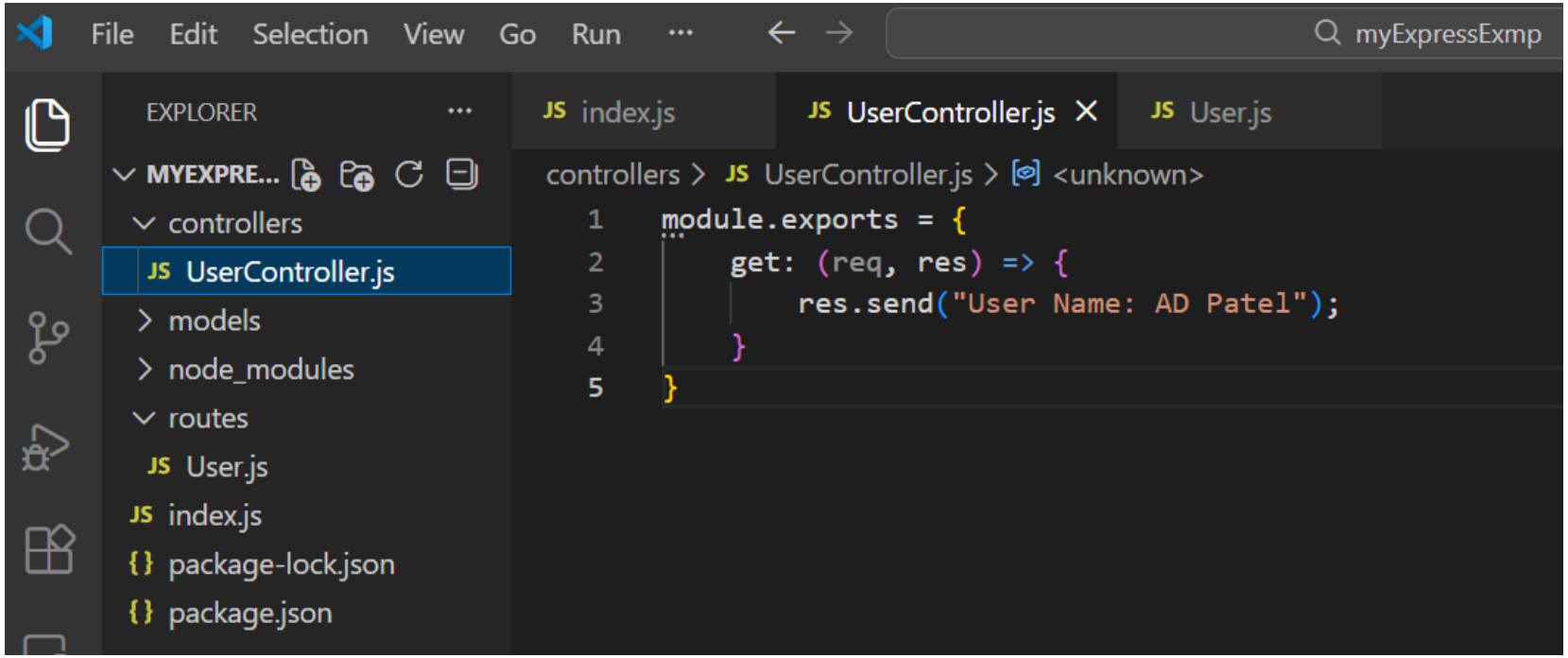
Example of MVC



Routs folder(User.js) it can be a model if database is used



Controller:



Benefits for Express.js Applications:

- **Separation of Concerns:** Code is organized and easier to manage.
- **Maintainability:** Changes in one component (e.g., database) minimally affect others.
- **Team Collaboration:** Different developers can work on models, views, and controllers simultaneously.
- **Testability:** Business logic in Models and Controllers can be tested independently of the UI.

Modules in Express.JS

math.js > ...

```
function add(a, b){
    return a + b;
}

function sub(a, b){
    return a - b;
}

module.exports = {add, sub};
```

index.js | math.js | UserController.js

index.js > ...

```
const math = require('./math');
console.log("Add is ", math.add(5, 6));

```

OR
(De-structuring)

index.js | math.js | UserController.js

index.js > ...

```
const {add, sub} = require('./math');
console.log("Add is ", add(5, 6));

```